

D212: Data Mining II

Task 1: Clustering Techniques

Hannah R. Bergman

WGU Email: hbergm3@wgu.edu

Part I: Research Question.....	2
A1. Research Question.....	2
A2. Goal of the Data Analysis.....	2
Part II: Technique Justification.....	2
B1. Clustering Technique & Expected Outcomes.....	2
How K-Means Clustering Works With Our Patient Dataset.....	2
Expected Outcomes of the Analysis.....	3
B2. Assumptions of the Chosen Clustering Method.....	3
B3. Python Packages & Libraries.....	3
Part III: Data Preparation.....	4
C1. Data Preprocessing Goal.....	4
C2. Variable Selection.....	4
Continuous (Numeric):.....	4
C3. Data Preparation Steps.....	5
Step 1. Detect & Handle Missing Values.....	5
Step 2. Outlier Detection & Handling.....	6
Step 3. Check & Fix Data Types.....	8
Step 4. Subset the Dataset.....	10
Step 5. Update Column Names.....	10
Step 6. Standardize the Dataset for K-Means Clustering.....	11
C4. Cleaned Dataset.....	11
Part IV: Data Analysis.....	12
D1. Optimal Number of Clusters & Elbow Method Description.....	12
Optimal Number of Clusters.....	12
How to Find the Optimal Number of Clusters using the Elbow Method.....	12
D2. Code.....	15
Part V: Data Summary & Implications.....	15
E1. Quality of the Clusters.....	15
E2. Results & Implications of the Clustering Analysis.....	16
Results.....	16
Implications.....	17
E3. Limitation of the Analysis.....	17

E4. Recommended Course of Action.....	17
Part VI: Demonstration.....	18
F. Panopto Video Recording.....	18
G. Data & Code Sources.....	18
H. References.....	18

Part I: Research Question

A1. Research Question

Can we identify distinct clusters of patients based on their age, income, vitamin D levels, number of doctor visits, full meals eaten per day, initial days hospitalized, total charges, and additional charges to better understand different patient health profiles and care utilization patterns?

A2. Goal of the Data Analysis

The goal of this analysis will be to identify distinct groups of patients based on key demographic, behavioral, and cost-related characteristics (such as age, income, vitamin D levels, doctor visits, meals eaten, and hospitalization charges) using K-means clustering, in order to uncover patterns that can help our hospital tailor care strategies, optimize resource allocation, and improve long-term cost-effectiveness in patient relationship management.

Part II: Technique Justification

B1. Clustering Technique & Expected Outcomes

How K-Means Clustering Works With Our Patient Dataset

K-means clustering works by calculating the distance between each data point and a centroid, which is the center point of a cluster, calculated as the average of all data points within that cluster (Yse, 2023), to assign it to a cluster. The goal of this clustering technique is to identify the K number of groups in the dataset (Sharma, 2024).

In regards to our own research, K-means clustering will find patterns in the dataset by grouping patients who share similar care and demographic traits. We'll start by choosing features relevant to our research question, such as Age, Income, VitD_levels (vitamin d levels), Doc_visits (number of doctor visits), Full_meals_eaten (full meals eaten per day), Initial_days (initial number of days hospitalized), TotalCharge, and Additional_charges. These variables represent patient characteristics and cost-related factors relevant to our research question.

Next, because these features have different scales (e.g. income vs. vitamin D levels), we'll standardize the data so that no single variable dominates the clustering process. We'll choose a number K (the number of clusters) and our algorithm will randomly select K initial centroids

(center points). Each patient will be assigned to the nearest centroid (cluster) based on Euclidean distance between their data values and each centroid. The algorithm will then calculate the centroid of each cluster by averaging all patient values in that cluster, repeating the process of assigning and recalculating until the algorithm converges - meaning the cluster assignments no longer change or reach a minimum threshold of change (GeeksforGeeks, 2025).

Expected Outcomes of the Analysis

The algorithm will group patients into clusters with similar patterns in their age, income, hospital usage and costs. Each cluster will be interpreted and labeled with a profile (such as “High Cost, High-Need Patients” vs. “Low-Risk, Low-Cost Patients”). These profiles help provide a more strategic understanding of patient segments, which supports targeted interventions, personalized treatment approaches, and better cost management.

B2. Assumptions of the Chosen Clustering Method

K-means assumes that clusters are spherical and evenly sized in the feature space, meaning the data points in each cluster are grouped closely around the cluster center (centroid), and clusters are roughly similar in shape and density (GeeksforGeeks, 2023). Because of this presumption of spherical clusters, K-means is vulnerable to clusters that are elongated or non-spherical in shape.

B3. Python Packages & Libraries

For this analysis, I'll be using Python - a programming language well suited for data science tasks like this. Python offers an extensive collection of powerful libraries and packages specifically designed to make working with data easier, including:

pandas (pd): Offers a high-performance, user-friendly data structure (DataFrame) that mimics spreadsheets for easy, efficient data analysis and manipulation.

- **CategoricalDtype**: Enables the creation of pandas columns that accurately reflect the categorical or ordinal nature of our data.

NumPy (np): Allows us to easily perform more complex mathematical computations within our dataset.

Matplotlib (plt): Enables the creation of various plot types, such as line plots, scatter plots, histograms, and bar charts, for effective data visualization and model assessment.

- **Seaborn** (sns): Builds upon Matplotlib to help us produce more statistically-informed and aesthetically pleasing visualizations.

SciPy (scipy.stats) Used for performing probabilistic distributions and statistical operations.

Scikit-learn (sklearn): Provides access to essential tools for the machine learning aspect of our analysis, including:

- **preprocessing**: Allows us to further preprocess our data.

- **StandardScaler:** Standardize the data to scale our features so that each has a mean of 0 and standard deviation of 1.
- **KMeans:** Applies the K-means clustering algorithm central to our analysis, which will divide our data into K groups (clusters) based on similarity.
- **silhouette_score:** Calculates the silhouette score we'll use to evaluate how well each point fits into its assigned cluster compared to the other clusters, assessing the quality of our model.
- **PCA:** Reduces the dimensionality of our data by creating new components (features) that capture the most variance (information) in the data.

```
# Import the python packages & libraries
import pandas as pd
from pandas.api.types import import_categorical_dtype
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
from scipy import stats
from sklearn import preprocessing
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
```

Part III: Data Preparation

C1. Data Preprocessing Goal

One of the primary data preprocessing goals will be to standardize all continuous variables so that each feature contributes equally to the distance calculations used in clustering, ensuring that variables like income and total charges do not dominate the analysis over others. Since some of our features (e.g., Income, TotalCharge) have much larger scales than others (e.g., Age, VitD_levels), without this standardization, the algorithm might place more weight on high-scale features, producing biased cluster assignments. By scaling the features (using StandardScaler), we'll ensure that all patient characteristics are treated fairly in the clustering process, aligning with our research goal to group patients based on similar overall profiles, not just one or two dominant attributes.

C2. Variable Selection

Continuous (Numeric):

- Age (years)
- Income (USD)
- VitD_levels
- Doc_visits
- Initial_days (days)
- Full_meals_eaten
- TotalCharge (USD)
- Additional_charges (USD)

These variables are selected based on their relevance to our research question - identifying groups of patients with similar demographic, health, and cost-related characteristics. They're also continuous variables, which makes them suitable for K-means clustering.

C3. Data Preparation Steps

Step 1. Detect & Handle Missing Values

Check for and address any missing values in the dataset for our variables of interest.

Identification

- Utilize the `isnull()` function to check for missing values in our dataset.
- Employ the `sum()` method to return the total number of missing values in each column.

```
# Check for missing values in our dataset
df.isnull().sum()
```

```
CaseOrder      0
Customer_id    0
Interaction     0
UID            0
City           0
State          0
County         0
Zip            0
Lat            0
Lng           0
Population     0
Area           0
TimeZone       0
Job            0
Children       0
Age           0
Income         0
Marital        0
Gender         0
ReAdmis       0
VitD_levels   0
Doc_visits     0
Full_meals_eaten 0
vitD_supp     0
Soft_drink     0
Initial_admin  0
HighBlood     0
Stroke        0
Complication_risk 0
Overweight    0
Arthritis     0
Diabetes      0
Hyperlipidemia 0
BackPain     0
Anxiety       0
Allergic_rhinitis 0
Reflux_esophagitis 0
Asthma       0
Services      0
Initial_days  0
TotalCharge   0
Additional_charges 0
Item1        0
Item2        0
Item3        0
Item4        0
Item5        0
Item6        0
Item7        0
Item8        0
dtype: int64
```

Investigation

As we can see from the result above, there appears to be no missing values in our dataset.

Handling

The dataset does not have missing values for our variables of interest.

Step 2. Outlier Detection & Handling

Identify and address data points that significantly deviate from the expected pattern or range in our dataset.

Identification

Since we're working solely with continuous variables, we'll use the `describe()` method to identify any potential outliers in the variables relevant to our research question, focusing on detecting data points that may indicate data entry errors or those that deviate significantly from the expected distribution.

```
# Summary statistics for continuous variables to check for outliers|
continuous_summary = df[['Age', 'Income', 'VitD_levels', 'Doc_visits', 'Initial_days', 'Full_meals_eaten', 'TotalCha
continuous_summary
```

	Age	Income	VitD_levels	Doc_visits	Initial_days	Full_meals_eaten	TotalCharge	Additional_charges
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	53.511700	40490.495160	17.964262	5.012200	34.455299	1.001400	5312.172769	12934.528587
std	20.638538	28521.153293	2.017231	1.045734	26.309341	1.008117	2180.393838	6542.601544
min	18.000000	154.080000	9.806483	1.000000	1.001981	0.000000	1938.312067	3125.703000
25%	36.000000	19598.775000	16.626439	4.000000	7.896215	0.000000	3179.374015	7986.487755
50%	53.000000	33768.420000	17.951122	5.000000	35.836244	1.000000	5213.952000	11573.977735
75%	71.000000	54296.402500	19.347963	6.000000	61.161020	2.000000	7459.699750	15626.490000
max	89.000000	207249.100000	26.394449	9.000000	71.981490	7.000000	9180.728000	30566.070000

Investigation

Review results to determine the cause of any outliers - whether they're due to errors, anomalies, or legitimate extreme values.

Age: The absence of patients younger than 18 indicates that our facility either does not provide care to minors or that their data is not included in the current dataset. The maximum age of 89 aligns with typical human life expectancy.

Income: There's a significant difference between the lowest income of \$154.08 and the highest reported income of \$207,249.10. The wide disparity between the minimum and maximum incomes could be a sign of outliers, such as very high or very low earners. We'll therefore explore this further utilizing the `nlargest()` and `nsmallest()` methods to inspect the lowest and highest patient incomes.

```
# Check for outliers in our Income data by looking at the largest and smallest values
income_largest = df['Income'].nlargest(30)
income_smallest = df['Income'].nsmallest(30)

print("LOWEST PATIENT INCOMES (Low to High):\n", income_smallest.to_string(index=False))
print("\nHIGHEST PATIENT INCOMES (High to Low):\n", income_largest.to_string(index=False))
```

LOWEST PATIENT INCOMES (Low to High):

```
154.08
300.79
395.23
401.86
493.04
695.22
702.16
798.98
826.01
881.07
881.40
953.74
1048.43
1078.12
1196.72
1209.56
1277.08
1286.25
1301.34
1366.98
1380.70
1392.23
1402.22
1409.92
1453.90
1480.67
1482.85
1486.44
1528.80
1541.63
```

HIGHEST PATIENT INCOMES (High to Low):

```
207249.10
204542.41
203774.60
197675.00
197576.18
196915.60
194796.24
190110.80
189416.27
189129.92
186791.40
186752.00
183037.10
179543.70
178945.40
178470.63
174745.85
171680.20
171288.05
171031.20
170961.50
167286.62
167105.10
166702.70
166048.50
166017.39
165588.50
165560.97
165329.55
164123.69
```

Looking at the largest and smallest values for our income data, it appears we indeed have a substantial range, encompassing individuals from very low-income to quite high-income backgrounds. While the majority of individuals fall within a typical income range, a small subset of individuals with exceptionally high incomes may be identified as outliers. However, considering the

potential for significant income growth and the absence of other relevant factors like geographical location, these extreme values are likely legitimate and not indicative of data errors.

VitD_levels: Average Vit D level = ~17.96, Minimum = ~9.80, Maximum = ~26.39. There's no apparent outliers with this variable but the amount of precision should be reduced to improve readability.

Doc_visits: Average number of doctors visits = ~5 visits, Minimum = 1, Maximum = 9. There are no apparent outliers with this variable.

Inital_days: Average length of stay = ~34.5 days (with a standard deviation of ~26.3 days), Minimum = ~1 day, Maximum = ~72 days. There are no apparent outliers with this variable.

Full_meals_eaten: Average number of meals = ~1, Minimum = 0, Maximum = 7. There are no apparent outliers with this variable.

TotalCharge: Average total charge = ~\$5,312.17, Minimum = ~\$1,938.31, Maximum = \$9,180.73. While there's a slight difference between the lowest and highest charges, the wide disparity is likely just the difference in costs related to individual patient needs within the hospital.

Additional_charges: Average additional charges = ~\$12,934.53, Minimum = ~\$3,125.70, Maximum = \$30,566.07. While there's certainly a significant difference between the lowest and highest charges, the wide disparity is likely just the difference in costs related to individual patient needs within the hospital.

Handling

Given our requirement for continuous variables and the ethical implications of using income to influence patient care, we'll retain the original income values in the dataset. Although this may introduce analytical challenges, it aligns with the project's objectives and avoids potential biases in healthcare outcomes.

Step 3. Check & Fix Data Types

Verify the data type of each column to ensure data consistency and compatibility.

Identification

Determine the data type of each column using the dtype function.

```
# Check the data type of each column
df.dtypes
```

```
CaseOrder      int64
Customer_id     object
Interaction     object
UID            object
City           object
State          object
County         object
Zip            int64
Lat            float64
Lng            float64
Population     int64
Area           object
TimeZone       object
Job            object
Children       int64
Age            int64
Income         float64
Marital        object
Gender         object
ReAdmis        object
VitD_levels    float64
Doc_visits     int64
Full_meals_eaten int64
vitD_supp      int64
Soft_drink     object
Initial_admin  object
HighBlood     object
Stroke        object
Complication_risk object
Overweight    object
Arthritis     object
Diabetes      object
Hyperlipidemia object
BackPain      object
Anxiety       object
Allergic_rhinitis object
Reflux_esophagitis object
Asthma        object
Services      object
Initial_days   float64
TotalCharge    float64
Additional_charges float64
Item1          int64
Item2          int64
Item3          int64
Item4          int64
Item5          int64
Item6          int64
Item7          int64
Item8          int64
dtype: object
```

Verification

Checking if the data types align with the expected values and the requirements of the algorithm, it appears the dtypes of this dataset are suitable for our analysis.

Continuous Variables: Age (int), Income (float), VitD_levels (float), Doc_visits (int), Full_meals_eaten (int), Initial_days (float), TotalCharge (float), and Additional_charges (float) are appropriately defined as numerical data types, already, allowing for quantitative analysis.

Handling

Numeric Adjustment:

- Convert Income data from decimal format to whole numbers by removing fractional parts.

```
# Cast Income column data type from float to integer
df["Income"] = df["Income"].astype(int)
```

- Round down TotalCharge, Additonal_charges, and VitD_levels to two decimal places to better reflect dollar amounts and for improved readability.

```
# Reduce column precision down to 2 decimal places to better reflect USD pricing and improve readability
df["TotalCharge"] = df.TotalCharge.round(2)
df["Additional_charges"] = df.TotalCharge.round(2)
df["VitD_levels"] = df.VitD_levels.round(2)
```

Step 4. Subset the Dataset

To focus on the relevant variables for our model, we'll create a new dataset called `df_subset` that includes only the columns we plan to use as features. This will help streamline our analysis and improve computational efficiency.

```
# Subset the dataset, selecting only the columns relevant to our research question
relevant_columns = [
    "Age",
    "Income",
    "VitD_levels",
    "Doc_visits",
    "Initial_days",
    "Full_meals_eaten",
    "TotalCharge",
    "Additional_charges"
]

df_subset = df[relevant_columns]

# Preview the resulting dataframe
df_subset
```

	Age	Income	VitD_levels	Doc_visits	Initial_days	Full_meals_eaten	TotalCharge	Additional_charges
0	53	86575	19.14	6	10.585770	0	3726.70	3726.70
1	51	46805	18.94	4	15.129562	2	4193.19	4193.19
2	53	14370	18.06	4	4.772177	1	2434.23	2434.23
3	78	39741	16.58	4	1.714879	1	2127.83	2127.83
4	22	1209	17.44	5	1.254807	0	2113.07	2113.07
...	...	...	...	...	...	...	...	...
9995	25	45967	16.98	4	51.561220	2	6850.94	6850.94
9996	87	14983	18.18	5	68.668240	0	7741.69	7741.69
9997	45	65917	17.13	4	70.154180	2	8276.48	8276.48
9998	43	29702	19.91	5	63.356900	2	7644.48	7644.48
9999	70	62682	18.39	5	70.850590	0	7887.55	7887.55

10000 rows × 8 columns

Step 5. Update Column Names

Rename column labels to provide more descriptive and meaningful names that accurately reflect the data they contain. This will enhance data clarity and prevent misunderstandings when working with the dataset. Ensure that the new labels are consistent in style and format.

```
# Update column labels for better understanding and consistency
updated_columns = [
    "age",
    "income",
    "vitamin_d_level",
    "doctor_visits",
    "inital_days",
    "full_meals_eaten",
    "total_charges",
    "additional_charges"
]

df_subset = df_subset.set_axis(updated_columns, axis=1)

# Preview the resulting dataframe
df_subset
```

	age	income	vitamin_d_level	doctor_visits	inital_days	full_meals_eaten	total_charges	additional_charges
0	53	86575	19.14	6	10.585770	0	3726.70	3726.70
1	51	46805	18.94	4	15.129562	2	4193.19	4193.19
2	53	14370	18.06	4	4.772177	1	2434.23	2434.23
3	78	39741	16.58	4	1.714879	1	2127.83	2127.83
4	22	1209	17.44	5	1.254807	0	2113.07	2113.07
...	...	...	...	...	...	...	...	...
9995	25	45967	16.98	4	51.561220	2	6850.94	6850.94
9996	87	14983	18.18	5	68.668240	0	7741.69	7741.69
9997	45	65917	17.13	4	70.154180	2	8276.48	8276.48
9998	43	29702	19.91	5	63.356900	2	7644.48	7644.48
9999	70	62682	18.39	5	70.850590	0	7887.55	7887.55

10000 rows x 8 columns

Step 6. Standardize the Dataset for K-Means Clustering

As mentioned earlier, one of our primary data preprocessing goals is to standardize our continuous variables. For this, we'll use `StandardScaler` to standardize our features by removing the mean and scaling to unit variance to ensure all variables contribute equally to the clustering process. We'll save the standardized features in a NumPy array called `scaled_data`, which can then be used as input for K-means clustering.

```
# Standardize the dataset for K-Means clustering

# Initialize the scaler
scaler = StandardScaler()

# Fit the scaler to the data and transform it
scaled_data = scaler.fit_transform(df_subset)
```

C4. Cleaned Dataset

The prepared dataset, '**d212_task1_transformed_medical_clean.csv**', can be found within the main project directory and is also included as an attachment to this report.

```
# Save the transformed dataset as a CSV file
df_scaled.to_csv('d212_task1_transformed_medical_clean.csv', index=False)

print(f"The final cleaned and scaled dataset has been saved as:\n 'd212_task1_transformed_medical_clean.csv'")
```

The final cleaned and scaled dataset has been saved as:
'd212_task1_transformed_medical_clean.csv'

Part IV: Data Analysis

D1. Optimal Number of Clusters & Elbow Method Description

Optimal Number of Clusters

The optimal number of clusters in our dataset is 3 ($K=3$).

How to Find the Optimal Number of Clusters using the Elbow Method

To determine the optimal number of clusters, I followed the Elbow Method, which plots the number of clusters (K) versus the inertia (within-cluster sum of squares). This method helps us find the point at which adding more clusters doesn't significantly improve the model's ability to group the data - just like an "elbow" in a curve.

To start, we create an empty list called inertia to store the inertia value (within-cluster sum of squares) for each value of K (number of clusters).

```
# Create an empty list to store the inertia value for each value of K
inertia = []
```

Next, we define a range of cluster numbers (K) to test. We'll start with 1 to 10 clusters (increasing if we see the need to do so when we visualize our K values vs. inertia later), fitting a K-means model for each of these values to see how the performance (inertia) changes.

```
# Define range of cluster numbers (K) to test, starting with 1 to 10 clusters
k_range = range(1, 11)
```

The for loop goes through each K value in the range we defined (1 to 10), creating a KMeans model for the current value of k . Setting `n_init=10` allows the algorithm to run 10 times with different centroid seeds and picks the best result, providing more stable and accurate clustering.

```
for k in k_range:
    # Create a KMeans model for the current value of k
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
```

The KMeans model is then fitted on the scaled dataset we created earlier in section C3 (`scaled_data`), creating k clusters. During this step, the algorithm assigns data points to clusters and calculates the centroid positions.

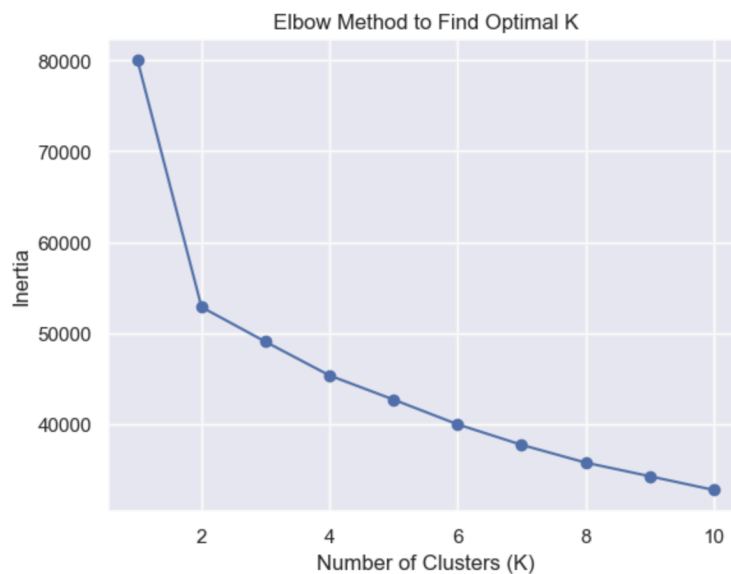
```
# Fit the Kmeans model on the scaled dataset
kmeans.fit(scaled_data)
```

The inertia value from the fitted model is then added to the inertia list. This value reflects how tight and compact clusters are for that K . Lower inertia is better, but adding more clusters typically lowers inertia, in general. With this, we're specifically looking for a point where it stops improving sharply (the "elbow").

```
# Add inertia value from the fitted model to the inertia list
inertia.append(kmeans.inertia_)
```

Finally, we plot a line chart of the K values vs. inertia to visualize the results, looking for the "elbow" point in the curve where the drop in inertia slows down, where the corresponding K value will be the optimal number of clusters for our data. Interpreting our chart, we can see that the "elbow" clearly appears around $K=3$. This is where the biggest drops taper off and clustering becomes less cost-effective beyond that so, therefore, the optimal number of clusters is 3.

```
# Plot a line chart of K values vs. inertia
plt.plot(k_range, inertia, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method to Find Optimal K')
plt.show()
```



To perform the cluster analysis, we define the optimal number of clusters (K) equal to 3, based on the Elbow Method analysis detailed above.

```
# Define optimal k
optimal_k = 3
```

We initialize a k-means clustering model utilizing scikit-learn's KMeans algorithm, setting the number of clusters (K=3) and number of times the k-means algorithm will run (n_init=10 - this will run the KMeans algorithm 10 times with different starting centroids, picking the best result, the one with the lowest inertia).

```
# Initialize KMeans model
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
```

We fit the KMeans model to the standardized dataset (scaled_data) we created in section C3, and the algorithm will assign each data point to the closest cluster centroid, predicting which cluster each data point belongs to. The result, stored as 'clusters', is a list of cluster labels (0, 1, or 2) assigned to each patient.

```
# Fit Kmeans model to standardized dataset
clusters = kmeans.fit_predict(scaled_data)
```

A new column called 'Cluster' is added to our DataFrame (df_subset) so each row (patient) includes their cluster assignment. This allows us to easily analyze and visualize the clusters.

```
# Add cluster labels to the original dataset
df_subset['Cluster'] = clusters
```

Using the newly created 'Cluster' column, we can group our dataset, where all rows belonging to the same cluster are grouped together. For each cluster, we calculate the average (mean) of the other columns, resulting in a summary table showing the average patient profile for each

cluster.

```
# Cluster summary table
cluster_summary = df_subset.groupby('Cluster').mean().round(2)

print("Cluster Summary Statistics:")
display(cluster_summary)
```

Cluster Summary Statistics:

	age	income	vitamin_d_level	doctor_visits	inital_days	full_meals_eaten	total_charges	additional_charges
Cluster								
0	53.80	40260.79	17.95	5.00	59.85	0.98	7394.01	7394.01
1	35.37	43079.86	17.84	4.97	9.34	0.96	3247.95	3247.95
2	70.99	38365.14	18.11	5.09	9.61	1.08	3280.36	3280.36

Based on our results, we can interpret each of these clusters and provide them with more meaningful labels:

Cluster 0:		Cluster 1:		Cluster 2:	
Feature	Value	Feature	Value	Feature	Value
Age	~53.8 years	Age	~35.4 years	Age	~71 years
Income	~\$40,261/yr	Income	~\$43,080/yr	Income	~\$38,365/yr
Initial Days	~59.85 days	Initial Days	~9.34 days	Initial Days	~9.61 days
Total Charges	~\$7,394.01	Total Charges	~\$3,247.95	Total Charges	~\$3,280.36
Additional Charges	~\$7,394.01	Additional Charges	~\$3,247.95	Additional Charges	~\$3,280.36

Cluster 0 comprises older middle-aged, middle-income patients with significantly prolonged hospital stays and consequently higher total charges. These '**High-Cost, Long-Stay**' patients represent a key priority for cost and risk management, as they demonstrate the highest resource utilization. Proactive care programs may be beneficial in reducing both resource consumption and patient costs within this cluster.

Cluster 1 consists of younger, higher-income patients with short hospital stays and lower overall costs. This '**Young, Low-Cost**' group may benefit significantly from preventative wellness initiatives, despite their low utilization of critical care services.

Cluster 2 comprises older, lower-income patients with moderate hospital utilization and costs. This '**Elderly, Moderate-Cost**' group necessitates targeted support to prevent escalation to high-cost care, similar to Cluster 0.

D2. Code

Please refer to the attached file '**d212_task1.ipynb**' for the code used to perform the clustering analysis.

Part V: Data Summary & Implications

E1. Quality of the Clusters

Cluster quality will be evaluated using the Silhouette Score, which measures the similarity of each point to its own cluster versus other clusters. The score ranges from -1 (poor clustering) to +1 (excellent clustering). A score closer to +1 indicates better cluster separation, which is optimal. Specifically, scores above 0.5 represent good clustering, 0.2 to 0.5 indicate moderate structure, and below 0.2 suggest poor clustering, implying potential overlap or randomness (Gultekin, 2023).

We'll implement scikit-learn's `silhouette_score` function to calculate this score for our model.

```
# Calculate silhouette score
sil_score = silhouette_score(scaled_data, clusters)
print(f"Silhouette Score: {sil_score:.2f}")

Silhouette Score: 0.22
```

The resulting Silhouette Score of 0.22 indicates the presence of cluster structure, but with limited distinctness. This suggests that a significant portion of patients are not well-separated by the clustering, and may reside near cluster boundaries.

This observation can be further validated through PCA visualization using scikit-learn.

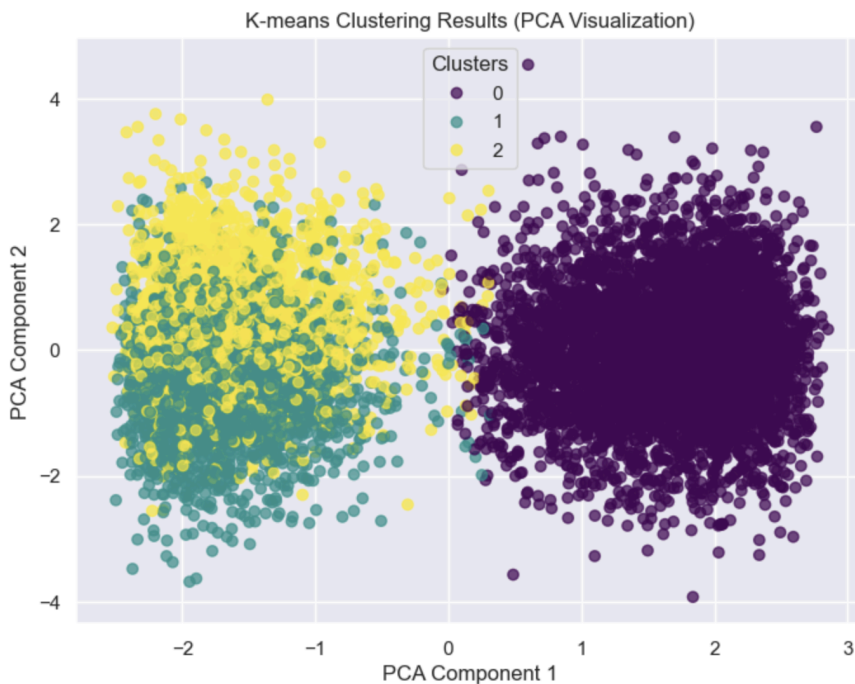
```
# Visualize clusters with PCA
pca = PCA(n_components=2)
pca_components = pca.fit_transform(scaled_data)
```

Plotting the clusters, we can see that Cluster 0 (Purple) is very dense and well-separated on the right side of the plot, indicating this cluster forms a cohesive group that is meaningfully different from Clusters 1 and 2. This is a strong example of good cluster quality.

However, on the left side of the plot, we also see that Cluster 1 (Teal) and Cluster 2 (Yellow) overlap heavily, indicating these clusters aren't well-separated in the feature space. Patients in Cluster 1 and Cluster 2 may have similar characteristics, making it more difficult for our K-means model to distinguish between them clearly.

This heavy overlap of Clusters 1 and 2 visually confirms what our Silhouette Score tells us - that our clusters are weakly defined overall, especially between these two groups. Cluster 0 contributes positively to the score, but having multiple overlapping clusters brings the score down considerably.

```
# Plot the clusters
plt.figure(figsize=(8, 6))
scatter = plt.scatter(pca_components[:, 0], pca_components[:, 1], c=clusters, cmap='viridis', alpha=0.7)
plt.title('K-means Clustering Results (PCA Visualization)')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.grid(True)
plt.legend(*scatter.legend_elements(), title="Clusters")
plt.show()
```



E2. Results & Implications of the Clustering Analysis

Results

Circling back to my initial research question, we successfully identified three distinct clusters of patients using K-means clustering (K=3). These clusters show meaningful differences in terms of age, income, hospital utilization (initial days, doctor visits), and cost of care (total and additional charges). This outcome was achieved by grouping patients based on the selected continuous features.

Our cluster profiles revealed one group (Cluster 0) represented high-cost, long-stay patients, potentially indicating chronic or complex cases. Another (Cluster 1) included younger, higher-income, low-cost patients, suggesting low-risk. A third (Cluster 2) comprised older, moderate-cost patients, indicating routine but aging care needs. In assessing the quality of these clusters, our Silhouette Score of 0.22 indicates weak structure, supported further by our PCA visualization - where we visibly saw two clusters significantly overlap.

Implications

Analysis of our patient clusters reveals that Cluster 0 patients (high-cost, long-stay) may drive increased costs and resource usage. To mitigate this, proactive care plans, chronic disease management, and targeted outreach could be implemented to reduce hospital stays and

charges. For Cluster 1 patients (young, low-cost), preventative care and wellness engagement programs could be offered to maintain their low-risk status. Cluster 2 patients (elderly, moderate-cost) may benefit from age-targeted support services, long-term care planning, fall-prevention, and nutrition programs.

Understanding these patient segments allows our hospital to tailor care delivery and improve cost-efficiency. By proactively addressing the needs of higher-cost patient groups, we can reduce overall long-term healthcare expenses and ensure proper resource planning (e.g., staffing, facilities, service offerings) to align with patient demand.

E3. Limitation of the Analysis

A key limitation of this analysis is that our cluster separation is weak, as reflected in our low silhouette score (0.22) and visual PCA overlap (specifically the overlapping of Clusters 1 and 2). The assumption of spherical, evenly sized clusters within the K-means algorithm may not be optimal for this dataset.

E4. Recommended Course of Action

In order to enhance the accuracy and quality of patient segmentation and create more meaningful groupings based on demographic, behavioral and cost-related characteristics, we may want to re-evaluate the number of clusters (K). Given the low silhouette score (0.22) and PCA plot indicating weak separation between some clusters, K=3 may be too simplistic. We can redo our cluster analysis, testing values between 2-6 and calculating the silhouette score for each to find the optimal balance of interpretability and structure. Our current feature set may not capture enough variance to differentiate patients clearly so we could also try including additional features, using correlation analysis or PCA explained variance to evaluate which features contribute most to variance. Additionally, we may want to consider testing alternative clustering algorithms. As discussed earlier, K-means assumes spherical, evenly sized clusters, which doesn't seem to fit our real-world patient data well. To address this, we can try experimenting with alternative clustering methods, such as opting for Hierarchical Clustering instead.

Part VI: Demonstration

F. Panopto Video Recording

Panopto video recording:

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=b602794a-a7d3-446a-ba52-b2b20121f6e1#>

G. Data & Code Sources

Western Governors University. (2023). *Medical Data and Dictionary Files* [Dataset]. Data Files and Associated Dictionary Files.

Retrieved March 20, 2025, from

<https://access.wgu.edu/ASP3/aap/content/g9rke9s0rlc9ejd92md0.html>

H. References

- Evidently AI. (2025, January 9). *Accuracy vs. Precision vs. Recall in Machine Learning: What's the Difference?* Classification Metrics Guide.
Retrieved March 19, 2025 from
<https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall>
- Frost, J. (2021, November 12). *Mean Squared Error (MSE)*. Statistics by Jim.
Retrieved March 19, 2025 from
<https://statisticsbyjim.com/regression/mean-squared-error-mse>
- GeeksforGeeks. (2025, January 15). *K Means Clustering - Introduction*.
Retrieved March 20, 2025 from
<https://www.geeksforgeeks.org/k-means-clustering-introduction/>
- GeeksforGeeks. (2023, December 9). *Demonstration of K-Means Assumptions*.
Retrieved March 20, 2025 from
<https://www.geeksforgeeks.org/demonstration-of-k-means-assumptions/>
- Gultekin, H. (2023, September 7). *What is a Silhouette Score?* Medium.
Retrieved March 21, 2025 from
<https://medium.com/@hazallgultekin/what-is-silhouette-score-f428fb39bf9a>
- Hunter, J., Dale, D., Firing, E., Droettboom, M., & Matplotlib Development Team. *Choosing Colormaps in Matplotlib*. Matplotlib.
Retrieved February 1, 2024, from
matplotlib.org/stable/users/explain/colors/colormaps.html#id4
- Sharma, N. (2024, April 15). *K-Means Clustering Explained*. [Blog Article].
Retrieved March 20, 2025 from
<https://neptune.ai/blog/k-means-clustering>
- Yse, D.L. (2023, June 30). *Introduction to K-Means Clustering*. Pinecone.
Retrieved March 20, 2025 from
<https://www.pinecone.io/learn/k-means-clustering/>